

06Lab

System modelling, a Programming Language approach: non-determinism, unbounded size, Petri Nets

Mirko Viroli

`mirko.viroli@unibo.it`

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
ALMA MATER STUDIORUM—Università di Bologna, Cesena

a.a. 2025/2026

One slide sum-up on Modelling

Concepts

- inspired by MDE, models are key to design (hence implementation), and verification (hence validation)
- behavioural models are systems per se, hence can be run/animated/simulated/analysed
- good engineering of behavioural models can be done at ProgLang level, and can lead to nice reusable meta-models

A stack of “models”: example

- Model: Petri-Net modelling of “readers and writers” (specific safety properties, design template)
- Meta-model: Petri Nets (captures parallelism, unbounded size, synchronisation, . . . , could be extended)
- Meta-meta-model: transition systems (captures non-deterministic state transition)

API organisation

- syntax of the model: via a DSL, or expressive API
- transition system: compiled from the DSL (states and transition rules)
- possibly adding analysis methods (carefully addressing loops/non-termination/divergence)

Starting point and goals

References

- <https://github.com/mviroli/asmd-public-models>

General goals

- get acquainted with our modelling stack, possibly improve it
- play with Petri Nets
- design new models

Operational Steps

Step 1 – Download and check

- Download the lab repository
- Import it in IntelliJ as an SBT project
- Get acquainted with all parts
 - ▶ `utils`: just the `MSet` abstraction
 - ▶ `modelling`: the actual API/DSL definition with `System`, `SystemAnalysis` (meta-meta-model); then `PetriNet` (meta-model)
 - ▶ `examples`: extensional example `SystemChannel`; intensional ad-hoc example `SystemMutualExclusion` and by Petri Nets `PNMutualExclusion`
 - ▶ `test`: checks and tests
- Check that all Scala tests and checks pass and why

Step 2 – “Implement” readers & writers, and play with it

- Code and do some analysis on the Readers & Writers Petri Net shown in slides, by cloning `PNMutualExclusion` – be sure to be technically excellent!
- Write some good test: what set of test cases would make one confident in the correctness of your design?
- What are the “defining” properties?
- If you drop loops (arcs from `p6` to `p1`, and `p7` to `p1`), will “validity” be easier to test/check?

Step 3 – Design a variation

- Code and do some analysis on a variation of the Readers & Writers Petri Net: it should be the minimal variation you can think of, such that if a process says it wants to read, it eventually (surely) does so. How would you show evidence that your design is right?
- Optionally, let an LLM of your choice help you in doing so: what would be a prompt (long?) prompt? what would be the role of testing?

Tasks

VERIFIER

- Consider Readers & Writers Petri Net. Add a test to check that in no path long at most 100 states mutual exclusion fails (no more than 1 writer, and no readers and writers together). Can you extract a small API for representing safety properties? What other properties can be extracted? How the boundness assumption can help?

ARTIST

- Create a variation/extension of PetriNet meta-model, with priorities: each transition is given a numerical priority, and no transition can fire if one with higher priority can fire. Add proper tests, such that we get enough confidence that our pretty new “abstraction” works as expected.
- Another interesting extension is “coloring”: tokens have a value attached, transitions have preconditions that also include predicates on those colors, and have functions to produce outgoing tokens’ colors from incoming ones. Again, provide suitable tests.

CHECKER

- How would you use ScalaCheck to capture “validity” of the general design of Petri-Net meta-model? (e.g., can we check that transitions “work as expected” in a variety of incoming/outgoing/inhibitory arcs configurations?)
- How would you use ScalaCheck to capture “validity” of a specific Petri Net model, like Readers & Writers? (e.g., can we have ScalaCheck generate a set of possible marking evolutions, so we check are all safe?)

RUNNER

- How a Petri-net could be useful in practice? It can be the specification of a concurrent monitor for processes! Try to fill this gap, developing a Scala engine that takes a Petri Net, and accordingly coordinate a system of processes.

TOOLING

- The current API might be re-organised: can we generate/navigate all paths (even with loops) thanks to caching and lazy evaluation?
- Some proposed the non-determinism is an effect, and can hence be handled by a Monad: can this idea be used to refactor our meta-metamodel support?

PETRINET-LLM

- We know that LLMs/ChatGPT can arguably help in write/improve/complete/implement/reverse-engineer standard ProgLangs. But is it of help in designing Petri Nets? Does it truly “understand” the model? Does it understand our DSL by examples?